

From NAND to ALU

Building Digital Logic from First Principles

Research Problem Statement:

Can every digital computing operation be derived from a single universal logic gate — the NAND gate?

nand

inv

and

or / xor

add 16

ALU

Research Problem Statement

Research Question: Can all fundamental digital logic operations and arithmetic computations be constructed exclusively from NAND gates, using Boolean algebra as the mathematical foundation?

Mathematical Basis

Boolean algebra — a branch of mathematics dealing with variables that take values of TRUE (1) or FALSE (0).

Functional Completeness

A single gate (NAND) is sufficient to express every possible Boolean function — proven mathematically.

Hierarchical Construction

Complex circuits (ALU, adders, selectors) are systematically built by composing simpler gate-level abstractions.

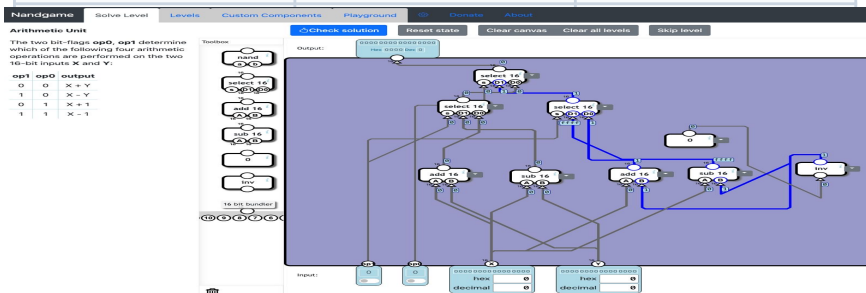
Level 1–2: NAND Gate & Inverter (NOT)

NAND Gate — Universal Primitive

Output is 0 only when BOTH inputs are 1. All other Boolean functions can be derived from NAND alone — making it functionally complete in Boolean algebra.

Truth Table: NAND(a, b)

a	b	Output
0	0	1
0	1	1
1	0	1



Inverter (NOT) — Built from NAND

Constructed by tying both NAND inputs together:

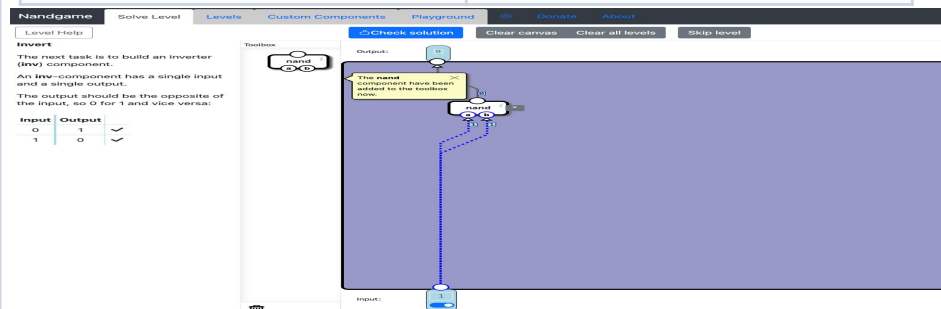
$$\text{inv}(a) = \text{NAND}(a, a)$$

Mathematically: $\neg a = a \text{ NAND } a$

This is our first derived gate — proof that NAND is sufficient.

Truth Table: inv(a)

Input (a)	Output
0	1
1	0



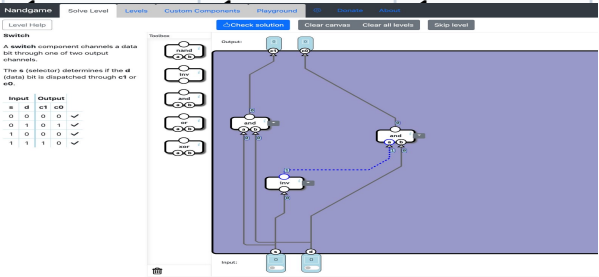
Level 3–5: AND, OR & XOR Gates

AND Gate

$$\text{AND}(a,b) = \text{inv}(\text{NAND}(a,b))$$

Output is 1 only when BOTH inputs are 1.
 Boolean: $a \wedge b$

a	b	Output
0	0	0
0	1	0
1	0	0

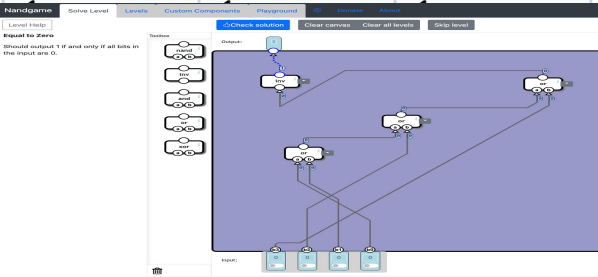


OR Gate

$$\text{OR}(a,b) = \text{NAND}(\neg a, \neg b)$$

Output is 1 when AT LEAST ONE input is 1.
 Boolean: $a \vee b$

a	b	Output
0	0	0
0	1	1
1	0	1

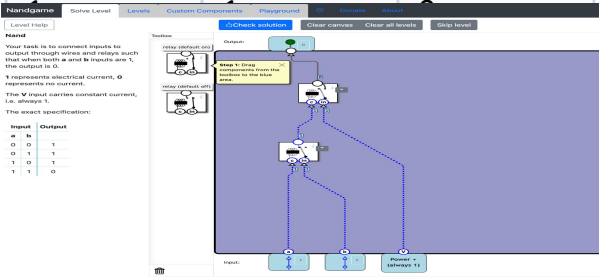


XOR Gate

$$\text{XOR} = \text{OR}(\text{AND}, \text{NAND})$$

Output is 1 when inputs DIFFER.
 Boolean: $a \oplus b$

a	b	Output
0	0	0
0	1	1
1	0	1



Level 6–7: Selector & Switch — Multiplexing Logic

Selector (Multiplexer)

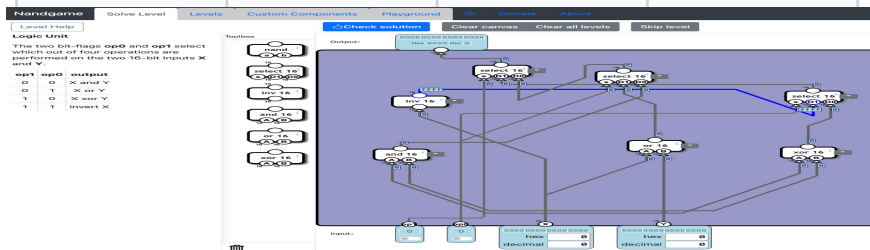
Selects one of two input bits for output.

- $s=0$: selects d_0 • $s=1$: selects d_1

Formula: $\text{out} = (s \text{ AND } d_1) \text{ OR } (\neg s \text{ AND } d_0)$

Foundational for building registers and ALU control paths.

	s	d1	d0	Output
0	0		0	0
0	1		0	0
0	0		1	1
0	1		1	1



Switch (Demultiplexer)

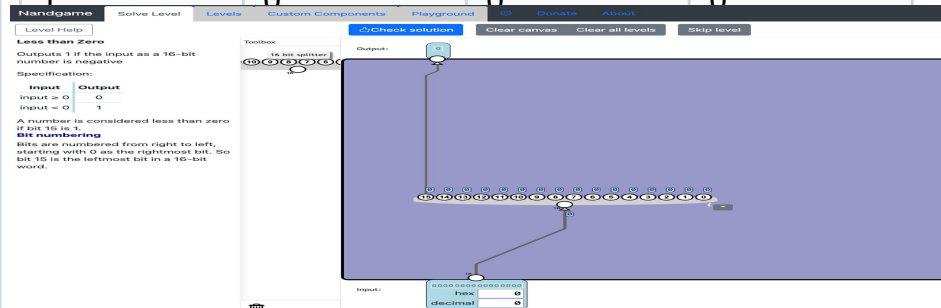
Routes a data bit d to one of two output channels.

- $s=0$: data $\rightarrow$ c_0 • $s=1$: data $\rightarrow$ c_1

Mathematically dual to the selector.

Implemented: $c_1 = s \text{ AND } d$, $c_0 = \neg s \text{ AND } d$

	s	d	c1	c0
0	0		0	0
0	1		0	1
1	0		0	0



Level 8–9: Comparators — Equal to Zero & Less than Zero

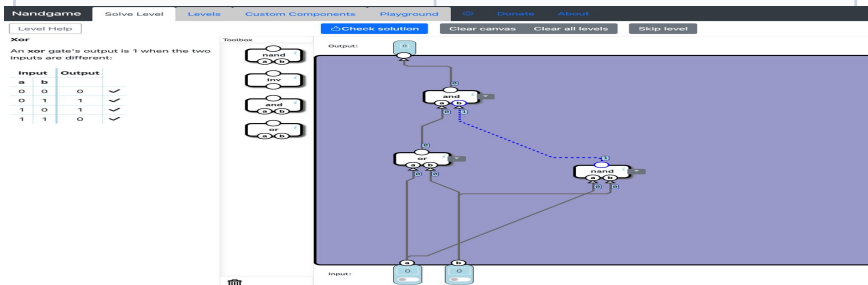
Equal to Zero (EQZ)

Outputs 1 iff ALL bits in the 4-bit input are 0.

out = NOT(b3 OR b2 OR b1 OR b0)

Chains OR gates, then inverts. Mathematically checks if the integer value equals zero.

Input	Output
0000	1
0001	0
1111	0



Less than Zero (LTZ)

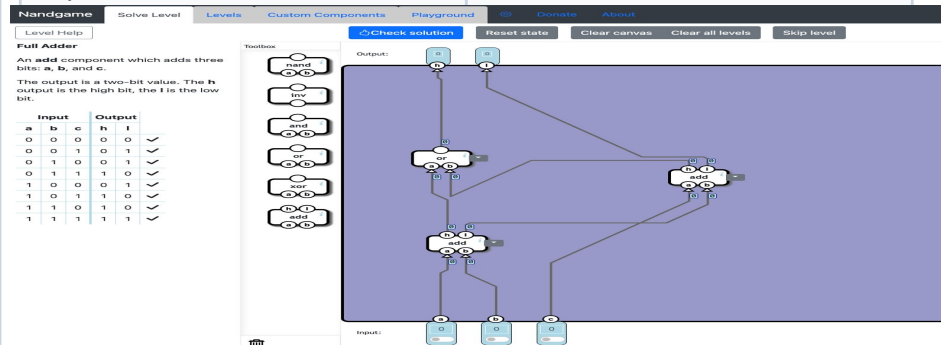
Outputs 1 if the 16-bit number is negative.

Uses Two's Complement: Bit 15 (MSB) is the sign bit.

- Bit 15 = 1 → number is negative
- Bit 15 = 0 → number is non-negative

Implemented by simply reading the MSB directly.

Input	Output
input ≥ 0	0
input < 0 (bit15=1)	1



Level 10–11: Half Adder & Full Adder

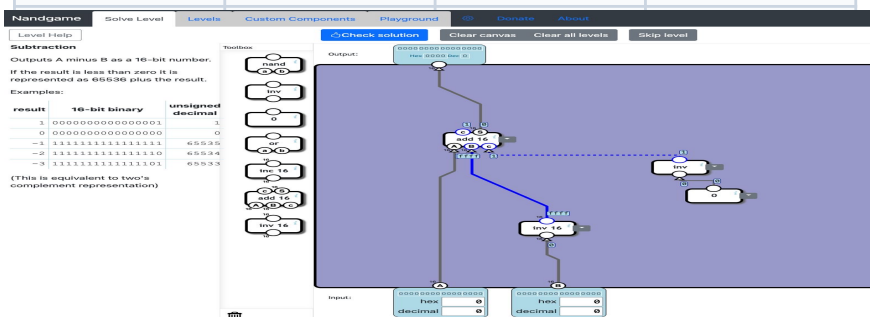
Half Adder

Adds two 1-bit numbers. 2-bit output:

- h (carry) = $a \text{ AND } b$
- l (sum) = $a \text{ XOR } b$

Represents binary addition mod 2.

	a	b	h (carry)	l (sum)
0	0	0	0	0
0	1	0	0	1
1	0	0	0	1



Full Adder

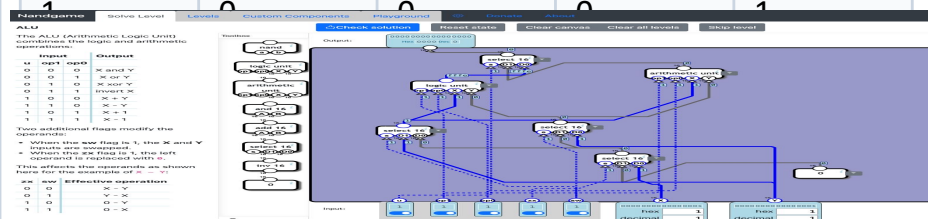
Adds THREE bits: a, b, carry-in c.

Two Half Adders + OR:

- $sum = (a \text{ XOR } b) \text{ XOR } c$
- $carry = (a \text{ AND } b) \text{ OR } (c \text{ AND } (a \text{ XOR } b))$

Enables chaining for multi-bit numbers.

	a	b	c	h (carry)	l (sum)
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	0	1
0	1	1	0	1	0
1	0	0	1	0	1
1	0	1	1	1	0
1	1	0	1	1	0
1	1	1	1	1	1

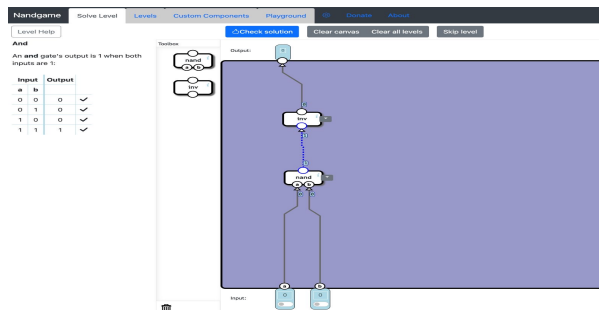


Level 12–14: Multi-bit Adder, Increment & Subtraction

Multi-bit Adder

Chains Full Adders for 16-bit addition.

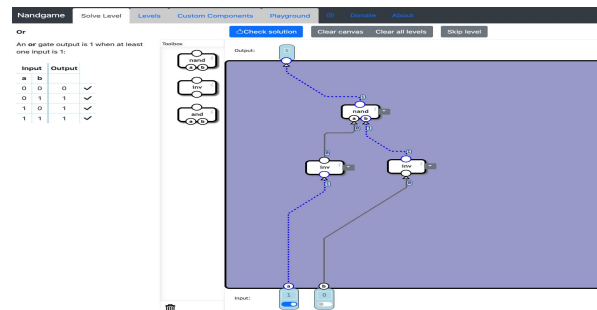
- Inputs: $a[15:0]$, $b[15:0]$
 - Ripple-carry architecture
 - Carry propagates bit by bit
- Basis of all addition hardware.



Increment (+1)

Adds constant 1 to a 16-bit number.

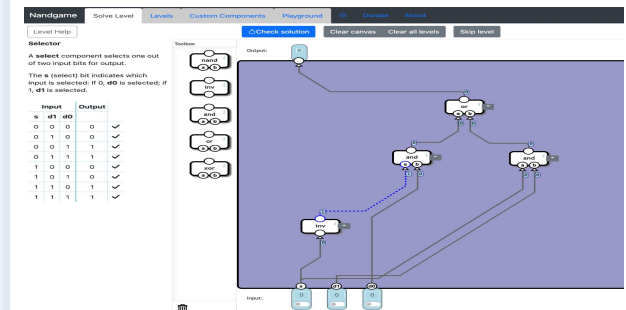
- B input fixed to 1
 - Carry-in set to 0
 - Overflow ignored (mod 65536)
- Critical for program counter.



Subtraction ($A - B$)

Exploits Two's Complement:

- $A - B = A + (\neg B) + 1$
- Invert B using inv16
- Add $A + \text{inv}(B) + 1$
- Same hardware as addition!



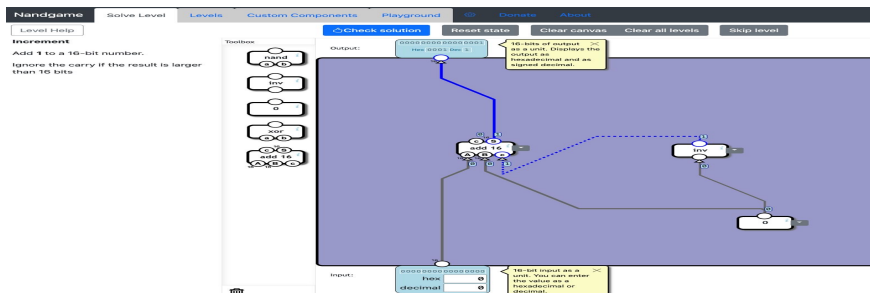
Level 15–16: Logic Unit & Arithmetic Unit

Logic Unit

Selects one of four 16-bit logic operations on inputs X and Y, controlled by op1 and op0:

	op1	op0	Operation
0	0		X AND Y
0	1		X OR Y
1	0		X XOR Y
1	1		invert X

Uses 16-bit parallel gates + select16 multiplexers.

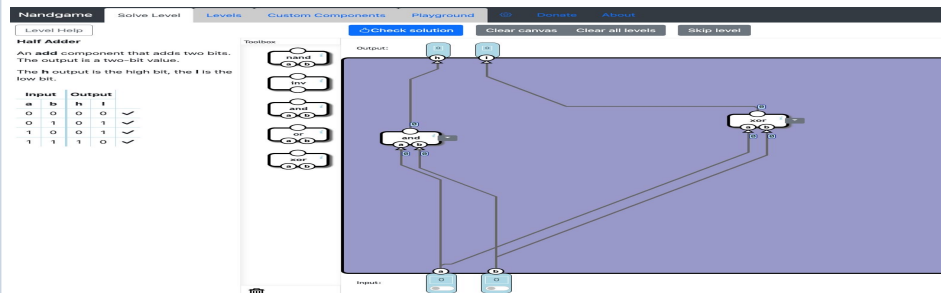


Arithmetic Unit

Selects one of four 16-bit arithmetic operations on inputs X and Y:

	op1	op0	Operation
0	0		X + Y
1	0		X - Y
0	1		X + 1
1	1		X - 1

Subtraction via Two's complement — same hardware, no extra cost!



The ALU

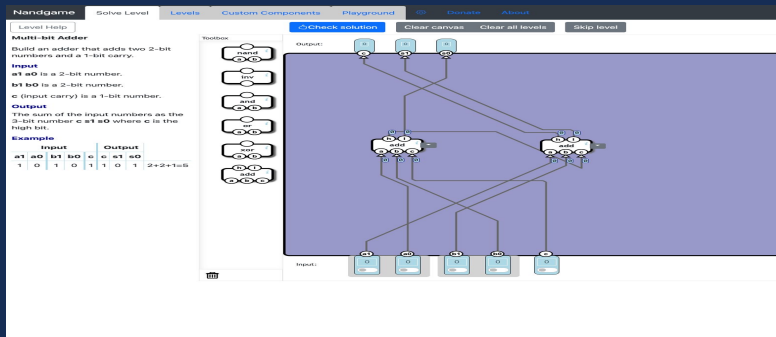
Arithmetic Logic Unit — Level 17

Combines Logic Unit + Arithmetic Unit.
Controlled by: u, op1, op0, zx, sw:

u op1 op0 Operation

u=0: AND · OR · XOR · invert X
u=1 op=00: $X + Y$ (addition)
u=1 op=10: $X - Y$ (subtraction)
u=1 op=01: $X + 1$ (increment)
u=1 op=11: $X - 1$ (decrement)

Flags: sw=1 → swap X,Y | zx=1 → replace left with 0



Research Conclusions

01

NAND is Functionally Complete — every Boolean function can be derived from NAND gates alone, as proven through systematic gate construction.

02

Hierarchical Abstraction — gates → adders → arithmetic units → ALU mirrors mathematical induction and modular proof strategies.

03

Two's Complement Elegance — subtraction, negation, and sign detection share identical hardware, a profound mathematical efficiency.

04

Boolean Algebra ↔ Hardware — every circuit level has a direct mathematical counterpart, validating the theoretical foundation of digital logic.

From a single NAND gate — an entire computing machine can be constructed. Q.E.D.